

《操作系统》

lab3：中断与中断处理流程

一. 实验要求

练习1：完善中断处理（需要编程）

请编程完善trap.c中的中断处理函数trap，在对时钟中断进行处理的部分填写kern/trap/trap.c函数中处理时钟中断的部分，使操作系统每遇到100次时钟中断后，调用print_ticks子程序，向屏幕上打印一行文字“100 ticks”，在打印完10行后调用sbi.h中的shut_down()函数关机。

要求完成问题1提出的相关函数实现，提交改进后的源代码包（可以编译执行），并在实验报告中简要说明实现过程和定时器中断中断处理的流程。实现要求的部分代码后，运行整个系统，大约每1秒会输出一行“100 ticks”，输出10行。

扩展练习 Challenge1：描述与理解中断流程

回答：描述ucore中处理中断异常的流程（从异常的产生开始），其中mov a0, sp的目的是什么？SAVE_ALL中寄存器保存在栈中的位置是什么确定的？对于任何中断，__alltraps 中都需要保存所有寄存器吗？请说明理由。

扩增练习 Challenge2：理解上下文切换机制

回答：在trapentry.S中汇编代码 csw sscratch, sp; crrw s0, sscratch, x0实现了什么操作，目的是什么？save all里面保存了stval scause这些csr，而在restore all里面却不还原它们？那这样store的意义何在呢？

扩展练习Challenge3：完善异常中断

编程完善在触发一条非法指令异常和断点异常，在 kern/trap/trap.c的异常处理函数中捕获，并对其进行处理，简单输出异常类型和异常指令触发地址，即“Illegal instruction caught at 0x(地址)”，“ebreak caught at 0x (地址)”与“Exception type:Illegal instruction”，“Exception type: breakpoint”。

二. 实验流程

练习1

按照练习的要求，我们完成 `IRQ_S_TIMER` 部分的补充，具体代码如下：

```
case IRQ_S_TIMER:
    // "All bits besides SSIP and USIP in the sip register are
    // read-only." -- privileged spec1.9.1, 4.1.4, p59
    // In fact, call sbi_set_timer will clear STIP, or you can clear it
    // directly.
    // cprintf("Supervisor timer interrupt\n");
    /* LAB3 EXERCISE1 YOUR CODE : */
    /* (1)设置下次时钟中断- clock_set_next_event()
    * (2)计数器(ticks)加一
    * (3)当计数器加到100的时候，我们会输出一个`100ticks`表示我们触发了100次时钟中断，同时打印
    次数(num)加一
    * (4)判断打印次数，当打印次数为10时，调用<sbi.h>中的关机函数关机
    */
```

```

// 设置下一次时钟中断
clock_set_next_event();

// 时钟中断计数器加一
ticks++;

// 每100次时钟中断处理一次
if (ticks >= TICK_NUM) {
    // 输出"100 ticks"
    print_ticks();

    // 重置计数器
    ticks = 0;

    // 打印次数计数
    static int print_count = 0;
    print_count++;

    // 打印10次后关机
    if (print_count >= 10) {
        sbi_shutdown();
    }
}
break;

```

然后我们输入 `make` 编译，注意这里不能使用过高版本的qemu，否则会报错，接着，我们输入 `make qemu`，得到输出结果：

```

PS D:\os-riscv\labcode\lab3> ws1
wey@wey:/mnt/d/os-riscv/labcode/lab3$ make clean
rm -f -r obj bin
wey@wey:/mnt/d/os-riscv/labcode/lab3$ make

+ cc kern/init/entry.S
+ cc kern/init/init.c
+ cc kern/libs/stdio.c
+ cc kern/debug/kdebug.c
+ cc kern/debug/kmonitor.c
+ cc kern/debug/panic.c
+ cc kern/driver/clock.c
+ cc kern/driver/console.c
+ cc kern/driver/dtb.c
+ cc kern/driver/intr.c
+ cc kern/trap/trap.c
+ cc kern/trap/trapentry.S
+ cc kern/mm/best_fit_pmm.c
+ cc kern/mm/default_pmm.c
+ cc kern/mm/pmm.c
+ cc libs/printfmt.c
+ cc libs/readline.c
+ cc libs/sbi.c
+ cc libs/string.c

```



```
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
```

扩展练习 Challenge1

ucore 中处理中断 / 异常的流程:

异常产生: 当外部中断或同步异常发生时, RISC-V 硬件会自动执行一系列操作: 将当前程序计数器保存到 sepc 寄存器, 以记录中断发生的位置; 将中断或异常的原因记录到 scause 寄存器, 用于区分具体的中断类型或异常类型; 将相关辅助信息保存到 stval 寄存器; 把当前特权级保存到 sstatus.SPP, 并切换到 S 模式; 同时禁用 S 模式中断, 并将原 SIE 值保存到 sstatus.SPIE; 最后跳转到 stvec 寄存器指向的中断入口点, 即 `__alltraps`。

中断入口处理 (trapentry.S): 首先执行 `SAVE_ALL` 宏, 该宏会将 32 个通用寄存器和 4 个关键 CSR (sstatus、sepc、stval、scause) 保存到栈中, 形成 struct trapframe 结构体。接着通过 `move a0, sp` 操作, 将栈顶指针作为参数传递给 C 函数 trap。随后调用 trap 函数, 从而进入 C 语言中断处理流程。

中断分发与处理 (trap.c): trap 函数会调用 trap_dispatch, 根据 scause 寄存器中的信息判断发生的是中断还是异常。若是中断, 由 interrupt_handler 进行处理, 例如时钟中断需要设置下一次中断并累加计数器; 若是异常, 则由 exception_handler 处理, 比如非法指令异常需要输出相关信息并更新 sepc 寄存器。

中断返回 (trapentry.S): 首先执行 `RESTORE_ALL` 宏, 从栈中恢复通用寄存器以及 sepc、sstatus 等 CSR 的值。之后执行 `sret` 指令, 该指令会将特权级恢复到中断前的级别, 将中断使能状态恢复为 sstatus.SPIE 的值, 并跳回 sepc 寄存器指向的地址, 使被中断的程序继续执行。

`mov a0, sp`的目的: 在 RISC-V 调用约定中, a0寄存器用于传递函数的第一个参数。`mov a0, sp`的作用是将栈顶指针作为参数传递给 C 函数trap, 使得trap函数能够访问中断发生时保存的寄存器和 CSR 信息, 从而进行中断类型判断和具体处理。

`SAVE_ALL`中寄存器在栈中的保存位置, 由struct trapframe结构体的成员定义顺序和`SAVE_ALL`宏的存储指令共同确定。首先, struct trapframe结构体明确规定了存储顺序: 先通过struct pushregs gpr按x0~x31的顺序存储 32 个通用寄存器, 之后依次存储status、epc、badvaddr、cause这 4 个 CSR, 整体占据 36 个uintptr_t的空间。其次, `SAVE_ALL`宏先通过`addi sp, sp, -36*REGBYTES`在栈上预留出对应大小的空间, 再按照结构体成员的偏移顺序执行存储指令, 比如`STORE x0, 0*REGBYTES(sp)`将x0存在栈顶偏移 0 个寄存器长度的位置, `STORE s1, 32*REGBYTES(sp)`将 sstatus的值存在偏移 32 个寄存器长度的位置, 确保栈中存储位置与结构体成员完全对应。

对于任何中断, `__alltraps`中都需要保存所有寄存器吗?

对于任何中断, `__alltraps`中都需要保存所有寄存器。原因在于中断是异步发生的事件, 被中断的程序可能处于任意执行状态, 无法提前预知哪些寄存器正在被使用。如果只保存部分寄存器, 中断处理程序在执行过程中可能会修改未保存的寄存器, 导致被中断程序恢复执行时, 因寄存器状态被破坏而出现逻辑错误。

扩增练习 Challenge2

`csw sscratch, sp`: 这条指令将当前栈指针的值写入sscratch寄存器。sscratch的主要用途是在中断发生时临时存储关键数据, 此处用于暂存中断前的sp值 —— 因为后续步骤会调整sp来预留栈空间, 若不提前保存, 原sp值会丢失。

`crrw s0, sscratch, x0`: 这条指令是“交换”操作: 将sscratch中保存的原sp值读取到s0寄存器, 同时将x0写入sscratch。通过s0寄存器将原sp值传递到栈中, 确保中断前的栈指针被完整记录; 将sscratch清零, 符合 ucore 的约定, 为后续中断类型判断提供依据。

SAVE ALL保存stval/scause但RESTORE ALL不还原的原因及保存意义:

scause用于区分中断类型或异常原因，是中断处理程序分发处理逻辑的核心依据。stval提供辅助信息，帮助处理程序定位问题。stval和scause仅对当前中断 / 异常有效，中断处理完成后，这些信息已无意义，被中断的程序执行时，并不依赖这两个寄存器的值，因此，SAVE_ALL保存它们是为了处理当前中断，而RESTORE_ALL不还原是因为它们对被中断程序的后续执行无影响，且无需保留。

扩展练习Challenge3

按照实验要求，补充完整代码：

```
case CAUSE_ILLEGAL_INSTRUCTION:
    // 非法指令异常处理
    fprintf("Exception type: Illegal instruction\n");
    fprintf("Illegal instruction caught at 0x%08x\n", tf->epc);
    // 标准指令通常是4字节
    tf->epc += 4;
    break;

case CAUSE_BREAKPOINT:
    // 断点异常处理
    fprintf("Exception type: breakpoint\n");
    fprintf("ebreak caught at 0x%08x\n", tf->epc);
    // ebreak 是2字节的压缩指令
    tf->epc += 2;
    break;
```

在init.c里添加测试示例:

```
cprintf("\n=== Testing Exception Handling ===\n");

asm volatile("ebreak"); // 断点异常

asm volatile(".word 0x00000000"); // 非法指令

cprintf("=== Exception Tests Completed ===\n\n");
```

最后得到输出结果：

```
wey@wey:/mnt/d/os-riscv/labcode/lab3$ make qemu
+ cc kern/init/init.c
+ cc kern/trap/trap.c
+ ld bin/kernel
riscv64-unknown-elf-objcopy bin/kernel --strip-all -o binary bin/ucore.img
```

OpenSBI v0.4 (Jul 2 2019 11:53:53)

/ _ \ / _ | _ \ _ |
 | | | | _ _ _ _ | (_ | |) | | |
 | | | | ' _ \ / _ \ ' _ \ \ _ \ | _ < | |
 | | _ | | |) | _ / | | | _) | |) | | | _
 \ _ _ / | . _ / \ _ | _ | | _ _ _ / | _ _ / _ _ |
 | |

|_|

Platform Name : QEMU Virt Machine
Platform HART Features : RV64ACDFIMSU
Platform Max HARTs : 8
Current Hart : 0
Firmware Base : 0x80000000
Firmware Size : 112 KB
Runtime SBI Version : 0.1

PMP0: 0x0000000080000000-0x000000008001ffff (A)
PMP1: 0x0000000000000000-0xffffffffffffffff (A,R,W,X)

DTB Init

HartID: 0

DTB Address: 0x82200000

Physical Memory from DTB:

Base: 0x0000000080000000

Size: 0x0000000080000000 (128 MB)

End: 0x0000000087ffffff

DTB init completed

(THU.CST) os is loading ...

Special kernel symbols:

entry 0xfffffffffc0200054 (virtual)

etext 0xfffffffffc0201f4c (virtual)

edata 0xfffffffffc0207028 (virtual)

end 0xfffffffffc02074a0 (virtual)

Kernel executable memory footprint: 30KB

memory management: default_pmm_manager

physical memory map:

memory: 0x0000000080000000, [0x0000000080000000, 0x0000000087ffffff].

check_alloc_page() succeeded!

satp virtual address: 0xfffffffffc0206000

satp physical address: 0x0000000080206000

=== Testing Exception Handling ===

Exception type: breakpoint

ebreak caught at 0xc02000a0

Exception type: Illegal instruction

Illegal instruction caught at 0xc02000a2

=== Exception Tests Completed ===

++ setup timer interrupts

100 ticks

100 ticks

100 ticks

100 ticks

100 ticks

100 ticks

100 ticks

100 ticks

100 ticks

100 ticks

